COMPUTER ENGINEERING

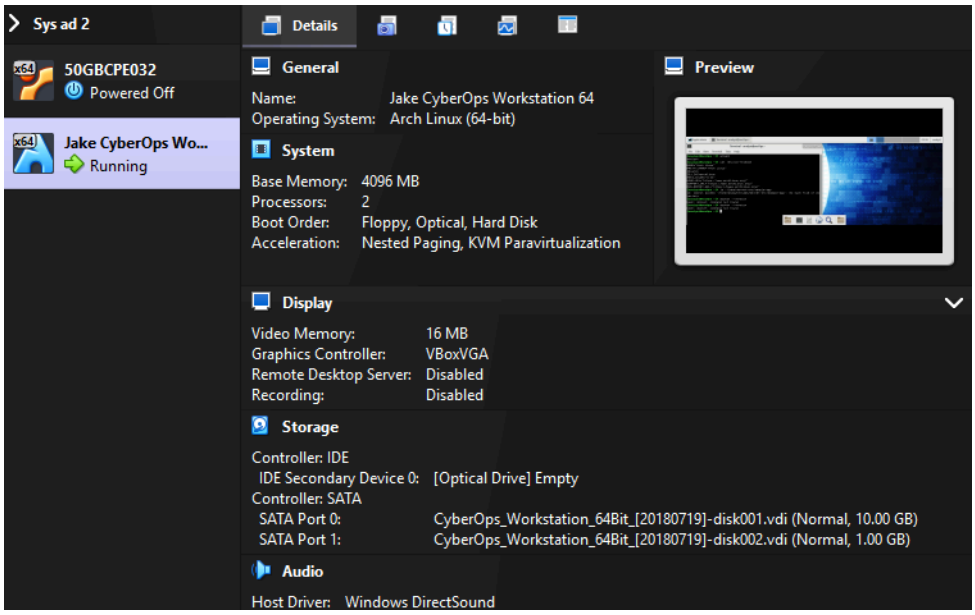
QUEZON CITY

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	18/10/2026	**DATE SUBMITTED:**	18/10/2026
NAME:	Damien Jake D. Gestiada	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12.1 **Build a Sample Web App in a Docker Container**

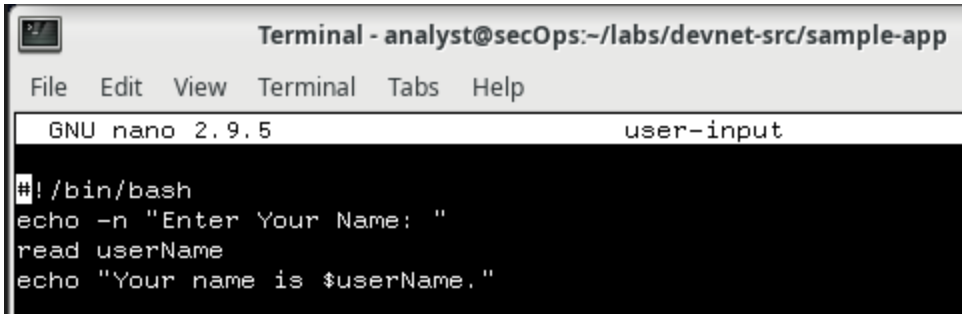
1. PROCEDURE OUTPUT

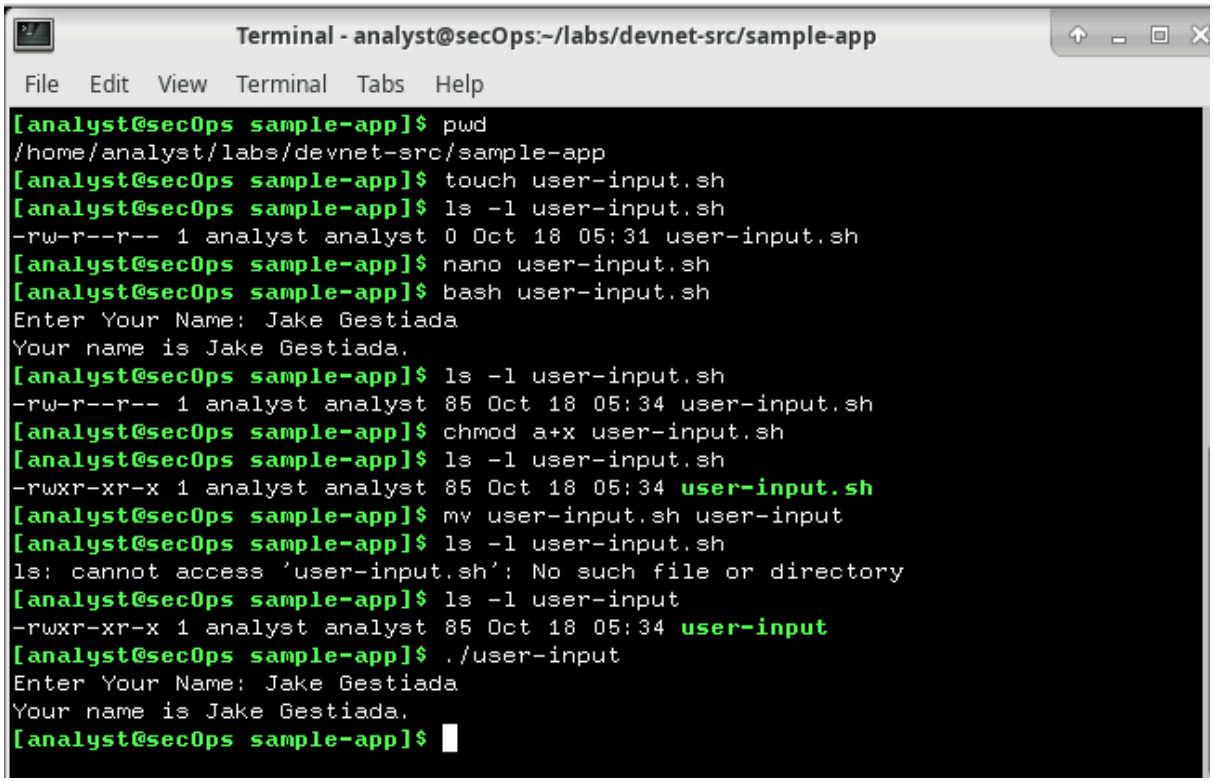
Part 1: Launch the DEVASC VM



Part 2: Create a Simple Bash Script

Step 1: Create an empty bash script file.
Step 2: Open the file in the nano text editor.
Step 3: Add the 'she-bang' to the top of the script.
Step 4: Add simple bash commands to the script.
Step 5: Exit nano and save your script.
Step 6: Run your script from the command line.
Step 7: Change the mode of the script to an executable file for all users.
Step 8: Rename the file to remove the .sh extension.
Step 9: Execute the script from the command line.
Step 10: Investigate other bash scripts.





Part 3: Create a Sample Web App

Step 1: Install Flask and open a port on the DEVASC VM firewall.
Step 2: Open the sample_app.py file.
Step 3: Add the commands to import methods from flask.
Step 4: Create an instance of the Flask class.
Step 5: Define a method to display the client IP address.
Step 6: Configure the app to run locally.
Step 7: Save and run your sample web app.
Step 8: Verify the server is running.
Step 9: Stop the server.

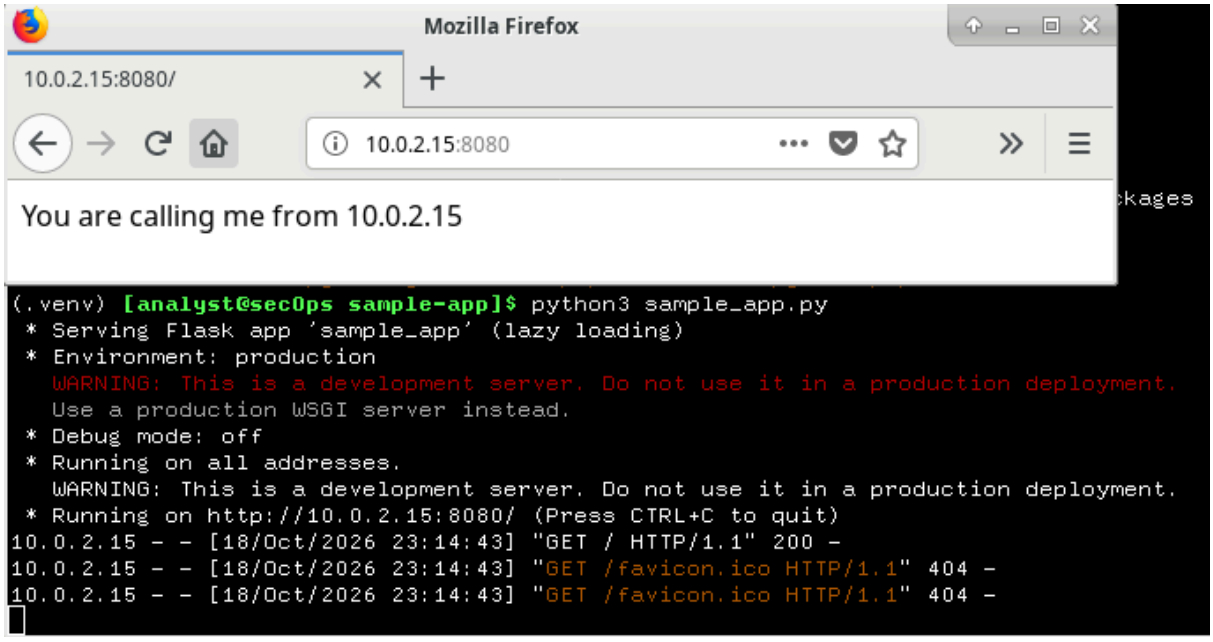
```
(.venv) [analyst@secOps sample-app]$ python3 -m pip show Flask
Name: Flask
Version: 2.0.3
Summary: A simple framework for building complex web applications.
Home-page: https://palletsprojects.com/p/flask
Author: Armin Ronacher
Author-email: armin.ronacher@active-4.com
License: BSD-3-Clause
Location: /home/analyst/labs/devnet-src/sample-app/.venv/lib/python3.6/site-packages
Requires: Jinja2, click, Werkzeug, itsdangerous
You are using pip version 9.0.3, however version 26.2.1 is available.
You should consider upgrading via the 'pip install --upgrade pip' command.
```

```
(.venv) [analyst@secOps sample-app]$ nano sample_app.py
(.venv) [analyst@secOps sample-app]$ cat sample_app.py
from flask import Flask
from flask import request

sample = Flask(__name__)

@sample.route("/")
def main():
    return "You are calling me from " + request.remote_addr + "\n"

if __name__ == "__main__":
    sample.run(host="0.0.0.0", port=8080)
(.venv) [analyst@secOps sample-app]$
```



Part 4: Configure the Web App to use Website Files

Step 1: Explore the directories that will be used by the web app.
Step 2: Update the Python code for the sample web app.
Step 3: Save and run your script.

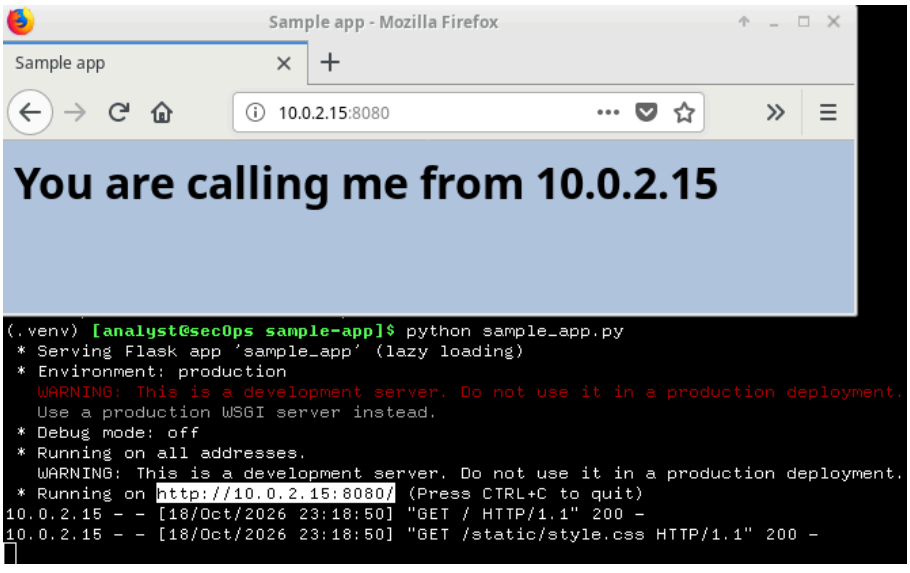
Step 4: Verify your program is running.
Step 5: Stop the server.

```
^C(.venv) [analyst@secOps sample-app]$ mkdir -p templates static
(.venv) [analyst@secOps sample-app]$ nano templates/index.html
(.venv) [analyst@secOps sample-app]$ cat templates/index.html
<html>
<head>
<title>Sample app</title>
<link rel="stylesheet" href="/static/style.css" />
</head>
<body>
<h1>You are calling me from {{request.remote_addr}}</h1>
</body>
</html>
(.venv) [analyst@secOps sample-app]$ nano static/style.css
(.venv) [analyst@secOps sample-app]$ cat static/style.css
body {background: lightsteelblue;}
(.venv) [analyst@secOps sample-app]$ nano sample_app.py
(.venv) [analyst@secOps sample-app]$ cat sample_app.py
from flask import Flask
from flask import request
from flask import render_template

sample = Flask(__name__)

@sample.route("/")
def main():
    return render_template("index.html")

if __name__ == "__main__":
    sample.run(host="0.0.0.0", port=8080)
```



Part 5: Create a Bash Script to Build and Run a Docker Container

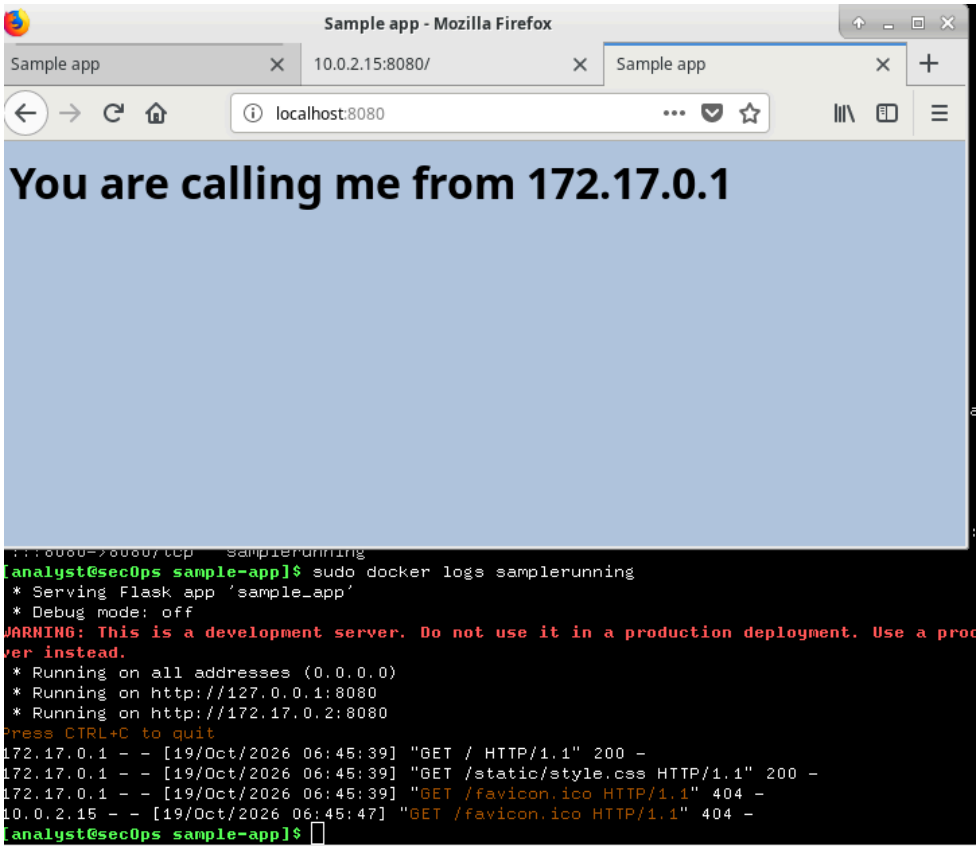
Step 1: Create temporary directories to store the website files.
Step 2: Copy the website directories and sample_app.py to the temporary directory.
Step 3: Create a Dockerfile.
Step 4: Build the Docker container.
Step 5: Start the container and verify it is running.
Step 6: Save your bash script.

```
[analyst@secOps sample-app]$ cat -n sample-app.sh
     1  #!/bin/bash
     2
     3  mkdir -p tempdir
     4  mkdir -p tempdir/templates
     5  mkdir -p tempdir/static
     6
     7  cp sample_app.py tempdir/.
     8  cp -r templates/. tempdir/templates/
     9  cp -r static/. tempdir/static/
    10
    11  cat > tempdir/Dockerfile <<'EOF'
    12  FROM python
    13  RUN pip install flask
    14  COPY ./static /home/myapp/static/
    15  COPY ./templates /home/myapp/templates/
    16  COPY sample_app.py /home/myapp/
    17  EXPOSE 8080
    18  CMD ["python3", "/home/myapp/sample_app.py"]
    19  EOF
    20
    21  cd tempdir
    22  docker build -t sampleapp .
    23
    24  docker run -t -d -p 8080:8080 --name samplerunning sampleapp
    25  docker ps -a
```

```
[analyst@secOps sample-app]$ sudo ./sample-app.sh
Sending build context to Docker daemon  6.144kB
Step 1/7 : FROM python
latest: Pulling from library/python
4846f2900730: Pull complete
62fe2e747f69: Pull complete
4dc3950125eb: Pull complete
b2eb680928d4: Pull complete
a6fa9d552f53: Pull complete
8f041c3070c7: Pull complete
f094b60471f5: Pull complete
Digest: sha256:e748956ccf3f50c9555f236753bbcd5f430d3937e665d3bd6922dd0e6e591a24
Status: Downloaded newer image for python:latest
 ---> f6b86d5d4f2d
Step 2/7 : RUN pip install flask
 ---> Running in b0e024a071bd
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.4-cp315-cp315-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 k
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.9-py3-none-any.whl.metadata (4.1 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.5.0-py3-none-any.whl (125 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.4-cp315-cp315-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (23 kB)
Downloading werkzeug-3.1.9-py3-none-any.whl (228 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask

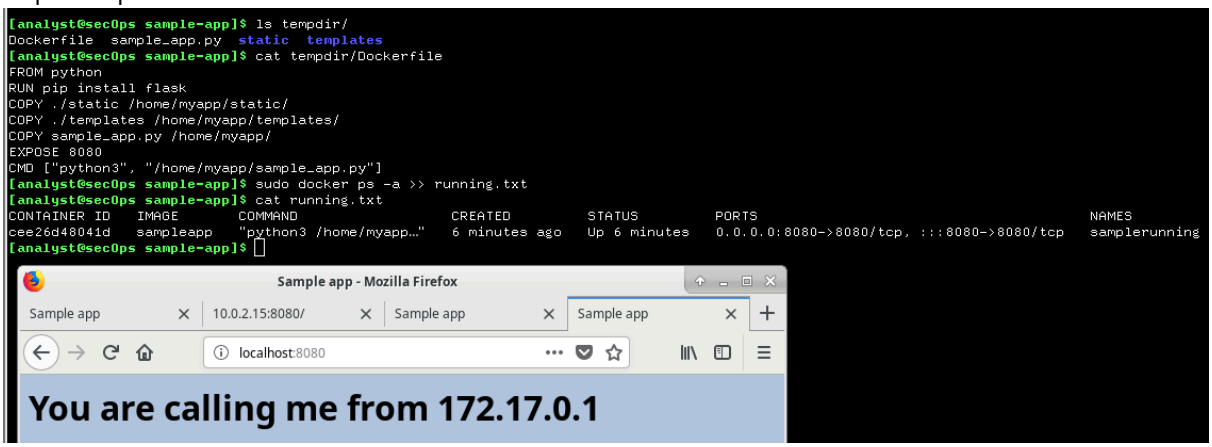
Successfully installed blinker-1.9.0 click-8.5.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.4 werkzeug-3.1.9
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package mana
ndering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use th
tion option if you know what you are doing and want to suppress this warning.
Removing intermediate container b0e024a071bd
 ---> 70ba90b33cdf
Step 3/7 : COPY ./static /home/myapp/static/
 ---> 54d1f84599b1
Step 4/7 : COPY ./templates /home/myapp/templates/
 ---> bffad6df7608
Step 5/7 : COPY sample_app.py /home/myapp/
```

```
Successfully installed blinker-1.9.0 click-8.5.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.4 werkzeug-3.1.9
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly r
ndering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-a
tion option if you know what you are doing and want to suppress this warning.
Removing intermediate container b0e024a071bd
 ---> 70ba90b33cdf
Step 3/7 : COPY ./static /home/myapp/static/
 ---> 54d1f84599b1
Step 4/7 : COPY ./templates /home/myapp/templates/
 ---> bffad6df7608
Step 5/7 : COPY sample_app.py /home/myapp/
 ---> 9bf9febff0e0
Step 6/7 : EXPOSE 8080
 ---> Running in a343bc0f022c
Removing intermediate container a343bc0f022c
 ---> fed070bd2fd9
Step 7/7 : CMD ["python3", "/home/myapp/sample_app.py"]
 ---> Running in 17a4643c635f
Removing intermediate container 17a4643c635f
 ---> 59b5b9514fe8
Successfully built 59b5b9514fe8
Successfully tagged sampleapp:latest
cee26d48041d86b0fc75de9e040d89add1f6059d07ea1bd5976a84eacc96b468
CONTAINER ID   IMAGE       COMMAND                  CREATED                  STATUS                  PORTS
  NAMES
cee26d48041d   sampleapp   "python3 /home/myapp…"   Less than a second ago   Up Less than a second   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp
  samplerunning
```



Part 6: Build, Run, and Verify the Docker Container

Step 1: Execute the bash script.
Step 2: Investigate the running Docker container and the web app.
Step 3: Access and explore the running container.
Step 4: Stop and remove the Docker container.



```
[analyst@secOps sample-app]$ curl http://localhost:8080
<html>
<head>
<title>Sample app</title>
<link rel="stylesheet" href="/static/style.css" />
</head>
<body>
<h1>You are calling me from 172.17.0.1</h1>
</body>
</html>[analyst@secOps sample-app]$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0f:9a:ba brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
       valid_lft 72162sec preferred_lft 72162sec
    inet6 fd17:625c:f037:2:a00:27ff:fe0f:9aba/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86309sec preferred_lft 14309sec
    inet6 fe80::a00:27ff:fe0f:9aba/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:de:d0:94:aa brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
    inet6 fe80::42:deff:fed0:94aa/64 scope link
       valid_lft forever preferred_lft forever
7: veth5c59453@if6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether 96:45:57:17:ea:5a brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::9445:57ff:fe17:ea5a/64 scope link
       valid_lft forever preferred_lft forever
```

```
[analyst@secOps sample-app]$ sudo docker exec -it samplerunning /bin/bash
root@cee26d48041d:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@cee26d48041d:/# ls /home/myapp/
sample_app.py  static  templates
root@cee26d48041d:/# exit
exit
[analyst@secOps sample-app]$ sudo docker stop samplerunning
samplerunning
[analyst@secOps sample-app]$ sudo docker ps -a
CONTAINER ID   IMAGE       COMMAND                  CREATED         STATUS                       PORTS     NAMES
cee26d48041d   sampleapp   "python3 /home/myapp…"   9 minutes ago   Exited (137) 7 seconds ago             samplerunning
[analyst@secOps sample-app]$ curl http://localhost:8080
curl: (7) Failed to connect to localhost port 8080: Connection refused
```

```
[analyst@secOps sample-app]$ sudo docker start samplerunning
samplerunning
[analyst@secOps sample-app]$ ^C
[analyst@secOps sample-app]$ sudo docker ps -a
CONTAINER ID   IMAGE       COMMAND                  CREATED         STATUS         PORTS                                       NAMES
cee26d48041d   sampleapp   "python3 /home/myapp…"   10 minutes ago   Up 6 seconds   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   samplerunning
[analyst@secOps sample-app]$ sudo docker stop samplerunning
samplerunning
[analyst@secOps sample-app]$ sudo docker rm samplerunning
samplerunning
[analyst@secOps sample-app]$ sudo docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
[analyst@secOps sample-app]$
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Conclusion:

- In this activity, I learned the procedure for constructing and carrying out a simple web app via Docker. I set up through a Bash script the relevant directories that I needed, put copies of my app code there, also created an empty Dockerfile. Moreover, I got familiar with such operations as Docker image building, launching a container, checking if the web service has been started, and finally I had a chance to exercise container inspection, stopp, reboot and deletion. In short, this exercise gave me an enhanced comprehension of the concept and operation of Docker, as well as its advantages as a technique for easier execution of web applications.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude and Grammary**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **For grammar correction and debug download resources**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.